SYSTEM GUIDE

Digital Thread Analytics

Programming Frameworks for End-to-End Data Science Lifecycle Management

A plain-language companion to the case-study report: what the project is, why it exists, what every term in it means, and how to run it.

Durvesh M.

Programming for Data Science (MACSE502)

School of Computer Science and Engineering

Vellore Institute of Technology

October 2026

Contents

1	The idea in one page
2	The problem, with a worked example
3	How the system is put together
4	The eleven artifact types
5	The eleven relation types
6	What happens when you run the pipeline
7	What happens when you make a prediction
8	Reading the results
9	Running the project
10	Glossary of every term used
11	Questions you should be ready for

1. The idea in one page

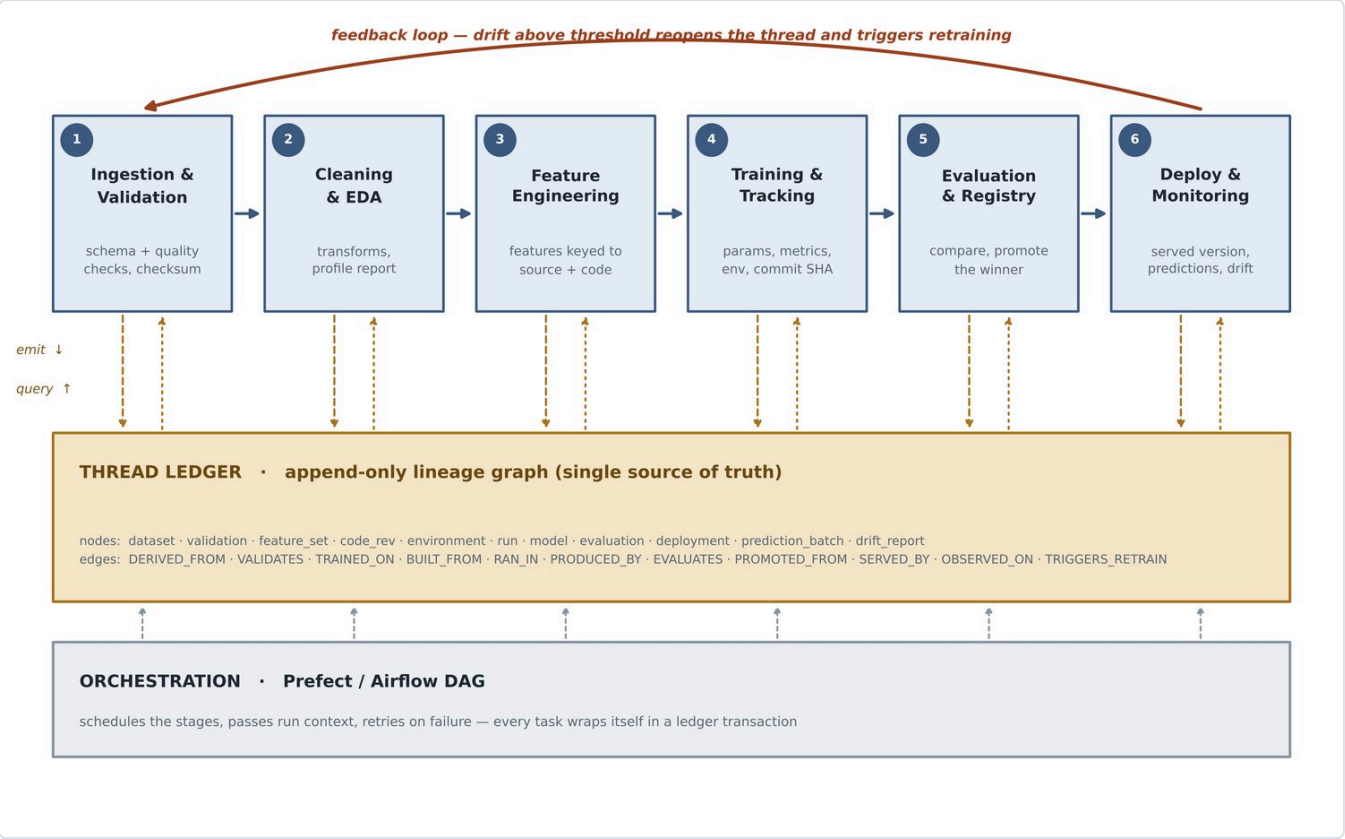
In manufacturing there is a concept called the **digital thread**. It means keeping one continuous, authoritative record that links every representation of a product across its whole life: the requirement that justified it, the design that satisfied the requirement, the simulation that tested the design, the machine that made the part, the inspection that passed it, and the service history that followed. The point is not that each of those things is recorded somewhere. The point is that they are *linked*, so you can start at any one of them and walk to any other.

This project asks a simple question: what if we did that for a machine learning model?

A model is a manufactured product too. Its raw material is data. Its manufacturing steps are cleaning and feature engineering. Its tooling is code, parameters and a computing environment. It is then put into service, where it is used to make decisions, and where it slowly stops working as the world it was trained on drifts away from the world it now sees.

Today each of those stages is recorded by a different tool, in a different vocabulary, in a different database. So a question that crosses stages — *which rows of which dataset, checked by which quality rules, transformed by which feature definitions, under which version of the code, produced the model that made this prediction, and how much has the data drifted since?* — cannot be answered by any of them.

> **What this project builds.** A small Python component called the **Thread Ledger**. Every stage of a data science pipeline writes into it what it read and what it produced. The result is a single graph covering the whole lifecycle that you can walk backwards to find causes, or forwards to find consequences. Around that ledger sits a working seven-stage pipeline, a prediction service, and a dashboard — all of which exist to demonstrate and measure the ledger, not the other way round.



The architecture. Six lifecycle stages, all writing to and reading from one ledger, with a feedback loop closing from monitoring back to ingestion.

2. The problem, with a worked example

Imagine a factory running the model built in this project. On Tuesday it predicts that a milling machine is 83% likely to fail. A maintenance engineer stops the line, inspects the machine, and finds nothing wrong. He asks a reasonable question: *why did it say that?*

In a conventional project

The data scientist opens the notebook. The notebook has been edited since Tuesday. The CSV it read has been overwritten by this week's extract. The model file has a timestamp but no record of which data produced it. The parameters are in a cell that has been re-run with different values. Nobody recorded which version of the cleaning code was in effect. The honest answer, after two days of work, is "probably because of tool wear, but I can't prove it."

In this project

The prediction came back with a deployment identifier attached. One query later:

```
$ python -m thread.query explain-model

model      model:random_forest@ce39825864cd
             algorithm: random_forest
             average_precision: 0.7354
             params: {'class_weight': 'balanced_subsample', 'max_depth': 12,
                      'min_samples_leaf': 2, 'n_estimators': 300}
             seed: 42

run          run:train:random_forest@train:random_forest-16ce56ea
run          run:features@features-e72cb462
run          run:clean@clean-edcf0b1b
run          run:ingest@ingest-b000c780
feature_set  feature_set:ai4i2020-features@ce39825864cd   {'rows': 10000}
dataset      dataset:ai4i2020-clean@48f4547469e1   {'columns': 7, 'rows': 10000}
dataset      dataset:ai4i2020-raw@155fe64a3108     {'columns': 14, 'rows': 10000}
code_rev     code_rev:repository@sha256:df089449d9c17c28
environment  environment:python-env@e16bf67bf27c
validation   validation:ai4i2020-expectations@155fe64a3108   passed 9 checks

answered in 0.4 ms across 9 nodes
```

That is the entire difference. Not a better model — the model is identical in both cases — but the ability to say, with evidence, what produced an answer.

3. How the system is put together

The repository has five parts. Only the first is the contribution; the rest exist to exercise and measure it.

FOLDER	WHAT IT CONTAINS	WHY IT EXISTS
`thread/`	The Thread Ledger: the graph store, fingerprinting, the query CLI, the graph renderer	The contribution of the project
`pipeline/`	Seven lifecycle stages, one Python function each, plus the flow that runs them in order	Demonstrates the ledger on a real lifecycle
`app/`	FastAPI backend and a dependency-free dashboard	Shows that lineage can travel with a live answer
`baseline/`	The same models, written as one ordinary script with no lineage	The control arm of the experiment
`experiments/`	The four-part comparative evaluation	Produces every number in the report
`tests/`	Twenty-five automated tests	Checks the properties the report claims

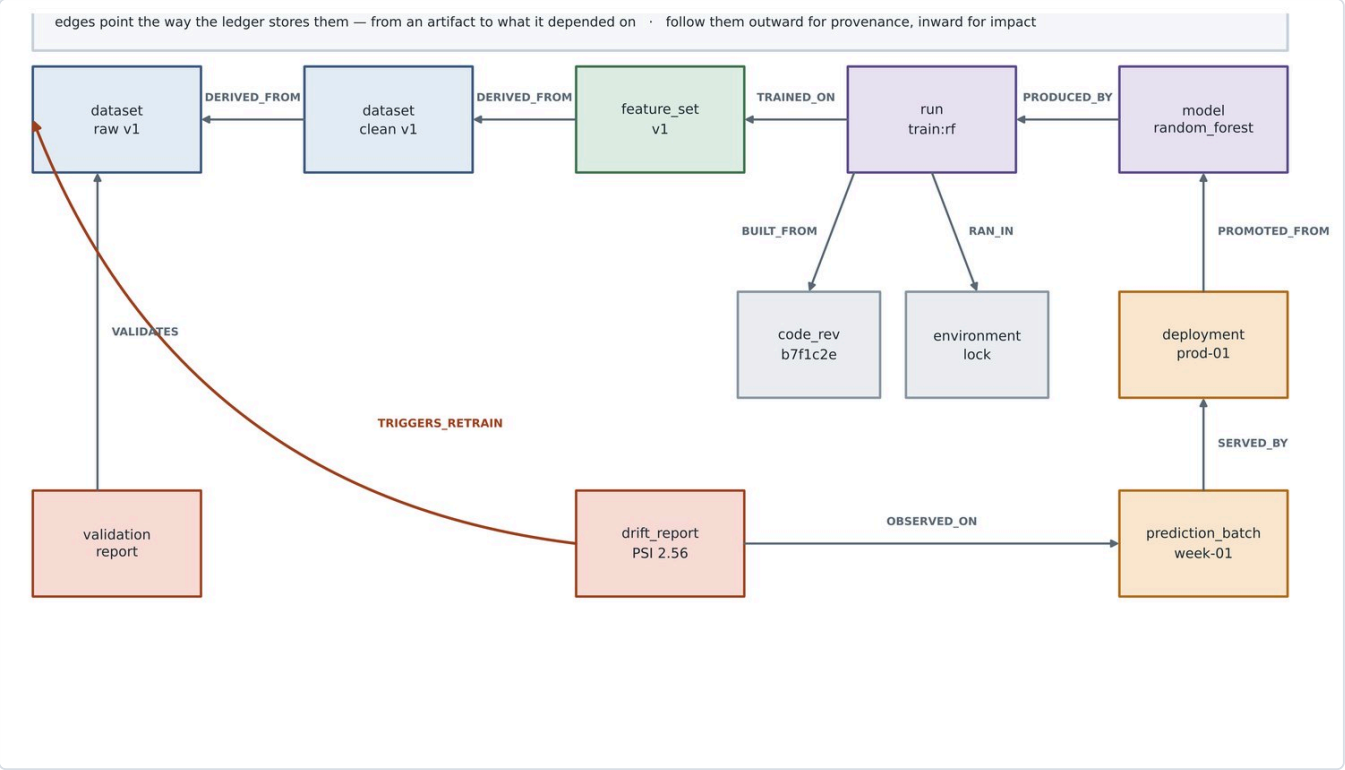
The three decisions that matter

Content-addressed identity. Each artifact's identifier is a hash of what it *is* — its type, name, version and a digest of its contents — rather than a number handed out by whatever created it. Re-run an unchanged stage and you get the same identifier. That is what makes the claim "this run is identical to that run" checkable rather than merely asserted.

Append-only, enforced by the database. The ledger's tables carry triggers that reject any attempt to update or delete a row. Not "we agree not to"; actually refused. A run's status is therefore never patched onto its record — a separate outcome record is written when the stage finishes, so even a crash leaves a trace.

One edge direction. Every link points backwards in time, from an artifact to whatever it depended on. This follows the W3C PROV standard. Because the direction never varies, one piece of traversal code is correct for the entire graph: follow links outwards to get history, inwards to get consequences.

> **Worth knowing for a viva.** An early version of this code mixed two edge directions. The traversal did not crash — it silently returned one node instead of the whole chain, and looked plausible. The lesson is that a lineage graph with inconsistent semantics fails quietly, which is the worst way for a system of record to fail.



The eleven artifact types and eleven relation types, shown over one complete pass of the pipeline. Arrows point the way the ledger stores them: towards the past.

4. The eleven artifact types

TYPE	WHAT IT REPRESENTS	KEY THINGS RECORDED ON IT
`dataset`	A specific version of a table of data	Row and column counts, SHA-256 digest, file path, origin
`validation`	The verdict of a data quality check suite	Each check, whether it passed, and the detail of any failure
`feature_set`	A versioned set of model inputs	The symbolic definition of every engineered feature
`code_rev`	The state of the source code at run time	Git commit, or a hash of the source tree if git is absent
`environment`	The interpreter and library versions	Python version, pandas, NumPy, scikit-learn versions
`run`	One execution of one pipeline stage	Parameters, seed, stage name
`model`	A trained model file	Algorithm, hyperparameters, all five metrics, file path
`evaluation`	A score, or the outcome of a run	Metrics, or status and duration for a stage outcome
`deployment`	A model promoted into service	Version, which model, the gate it cleared and the reason
`prediction_batch`	A set of predictions actually served	Row count, how many flagged, mean probability
`drift_report`	A measurement of distribution shift	PSI per feature, threshold, whether it drifted

5. The eleven relation types

RELATION	READS AS	EXAMPLE
`DERIVED_FROM`	this artifact was computed from that one	clean dataset → raw dataset
`VALIDATES`	this verdict concerns that dataset	validation report → raw dataset
`TRAINED_ON`	this run consumed that feature set	training run → feature set
`BUILT_FROM`	this run used that code revision	training run → code_rev
`RAN_IN`	this run executed in that environment	training run → environment
`PRODUCED_BY`	this artifact came out of that run	model → training run
`EVALUATES`	this score concerns that model	evaluation → model
`PROMOTED_FROM`	this deployment serves that model	deployment → model
`SERVED_BY`	these predictions came from that deployment	prediction batch → deployment
`OBSERVED_ON`	this drift was measured on that batch	drift report → prediction batch
`TRIGGERS_RETRAIN`	this drift sends that data back for retraining	drift report → raw dataset

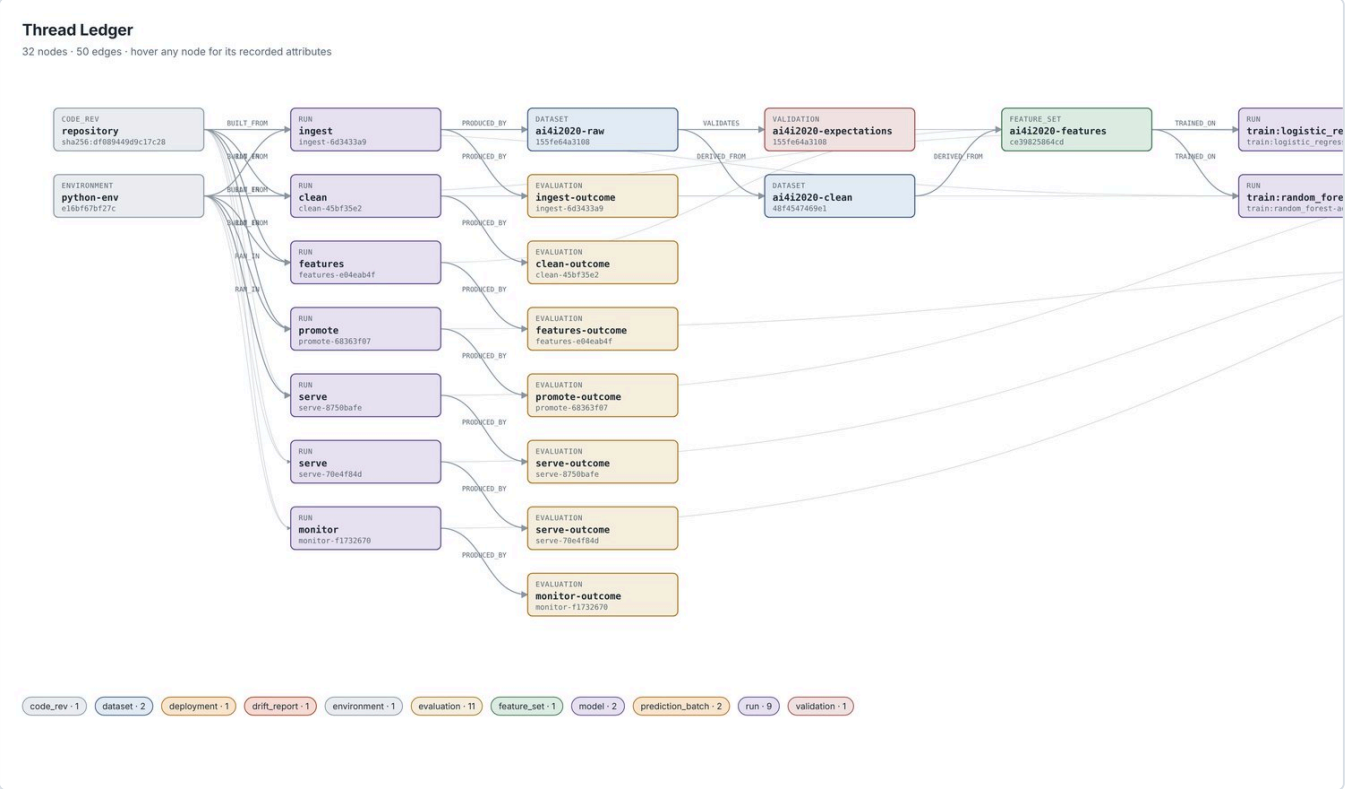
The last relation is the one that turns the lifecycle from a line into a loop. It is written automatically when the drift statistic crosses its threshold.

6. What happens when you run the pipeline

`python run.py pipeline` executes seven stages in order. Each opens a ledger transaction, does its work, records what it consumed and produced, and closes with an outcome record carrying its status and duration.

#	STAGE	WHAT IT DOES	WHAT IT WRITES TO THE LEDGER
1	Ingest & validate	Fetches the dataset, hashes it, runs nine quality checks	`dataset` (raw) and `validation`
2	Clean & profile	Drops identifiers and leakage columns, deduplicates, profiles	`dataset` (clean), linked to raw
3	Feature engineering	Builds three physics-derived features, one-hot encodes quality	`feature_set` with definitions
4	Train & track	Fits two candidate models on an identical split	Two `run`, two `model`, two `evaluation`
5	Evaluate & promote	Ranks on PR-AUC, applies the gate, promotes the winner	`deployment`
6	Serve	Scores a batch using the promoted model	`prediction_batch`
7	Monitor	Ages the batch, measures PSI, triggers retraining if needed	`drift_report` and the retrain link

The run finishes in about five seconds and leaves 32 artifacts and 50 links in a 60 KB file. It also writes `artifacts/thread.html`, a self-contained picture of the graph that can be opened in any browser years later without the framework installed.



The rendered ledger after one complete run, read left to right in the order things actually happened.

7. What happens when you make a prediction

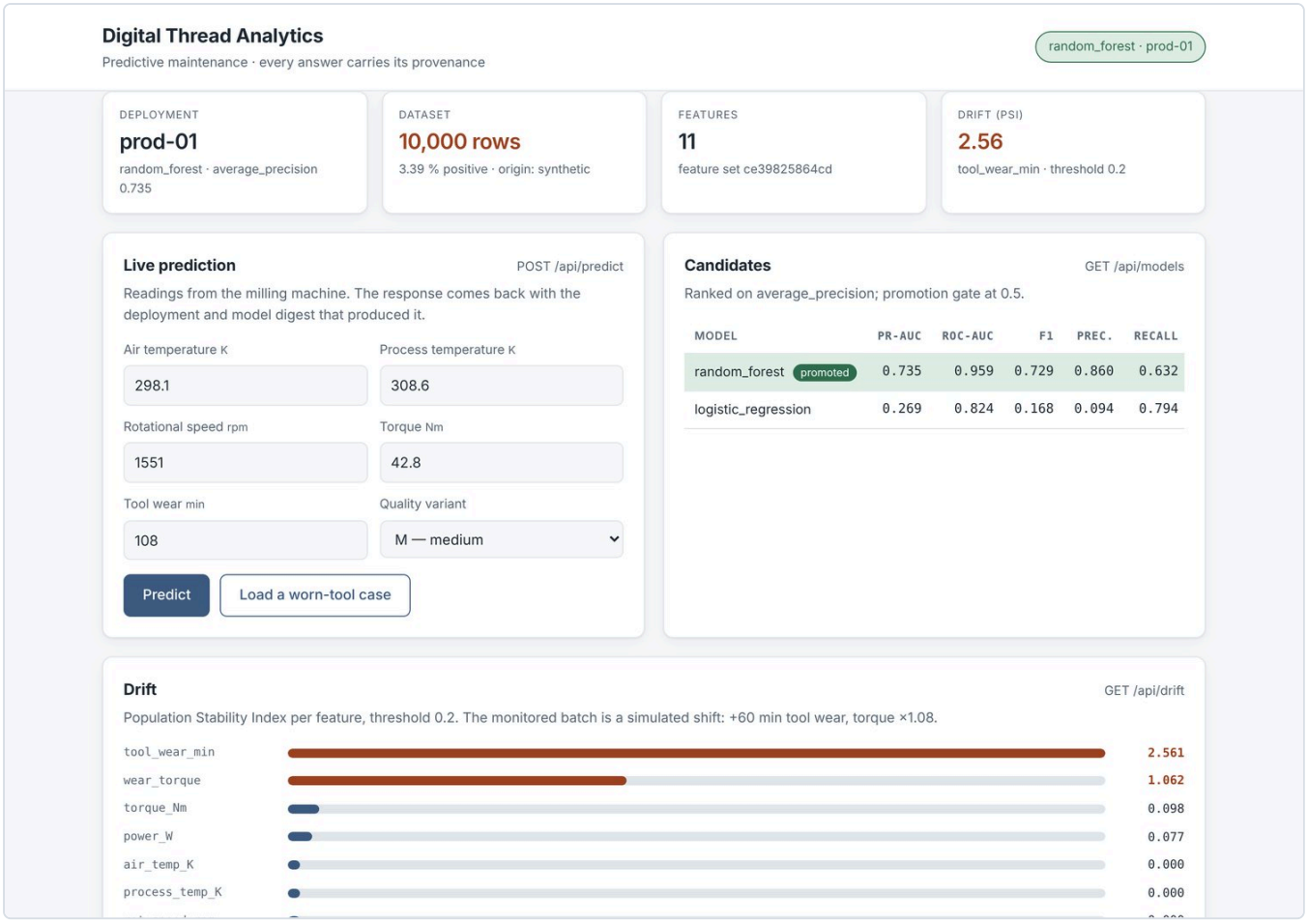
Start the service with `python run.py serve` and open `http://127.0.0.1:8000`. Enter a machine reading and press Predict. Three things happen that are worth understanding.

First, the features are recomputed on the serving path using the same definitions recorded in the feature set. This matters: a very common production bug is that training and serving compute a feature slightly differently, and the

model silently degrades. Because the definitions are stored as part of the artifact, the two cannot drift apart unnoticed.

Second, the response carries its own provenance — the deployment version, the model digest and the deployment's node identifier. The answer is not just a number; it knows where it came from.

Third, clicking *Trace this prediction back to raw data* turns that identifier into the full chain in a single further call, in well under a millisecond.



The dashboard. Everything shown is read from the ledger at request time; the backend keeps no state of its own, so re-running the pipeline is reflected without a restart.

8. Reading the results

`python run.py evaluate` runs the comparative study and prints five headline numbers. Here is what each one actually means.

RESULT	VALUE	HOW TO INTERPRET IT
Reproducibility rate	100% over 5 runs	Not "machine learning is deterministic". It means that with inputs, code and seed held constant, artifact identities are stable — so a future divergence would be *detected* instead of absorbed silently.
Lineage completeness	100% (6 of 6)	Every endpoint artifact has a path back to raw data. It does *not* prove the path is exhaustive — see the limitation on page 1 of Section 11.
Trace time	0.35–0.68 ms	Time to answer a provenance question. The baseline has no number here because it cannot answer at all, not because it is slow.
Instrumentation cost	1.9% of runtime	Time spent actually writing lineage. This is the honest cost of the ledger.
End-to-end overhead	+40.5%	The full difference against the baseline script. Most of it is persisting intermediate files that the baseline throws away — a precondition of traceability, but not the ledger's cost.

> **Why two overhead numbers?** Quoting only 1.9% would be self-serving; quoting only 40.5% would be misleading. The gap between them is artifact persistence. Also note that the pipeline is dominated by fitting a random forest in 3.06 seconds — on a realistic training workload the fixed cost of lineage becomes proportionally tiny. The durable figure is the absolute one: 0.073 seconds and 60 KB.

9. Running the project

INSTALL

```
pip install -r requirements.txt
```

THE FIVE COMMANDS YOU NEED

```
python run.py pipeline     # run all seven stages end to end
python run.py serve        # dashboard + API at http://127.0.0.1:8000
python run.py query        # trace the deployed model on the command line
python run.py evaluate     # the comparative study against the baseline
python run.py test         # 25 automated tests
```

OTHERS

```
python run.py stats        # census of the ledger
python run.py baseline     # the control arm, no lineage
python run.py export       # re-render artifacts/thread.html
python run.py clean        # delete generated artifacts
```

If something goes wrong

SYMPTOM	CAUSE	FIX
Dashboard says "no-ledger"	The pipeline has not been run	`python run.py pipeline`, then reload
Dataset tile shows "origin: synthetic"	The UCI download was unreachable, so a replica was generated	Check your network, delete `data/raw/`, re-run. See the warning below.
API tests skipped	No ledger exists yet	Run the pipeline first
`ModuleNotFoundError`	Dependencies not installed, or wrong directory	Run from the project root after installing requirements
Port 8000 already in use	A previous server is still running	Stop it, or edit the port in `run.py`

> **Before you submit anything.** If the dataset origin reads `synthetic`, the numbers came from a replica generated from AI4I 2020's published description, not from the real file. Download the real dataset, set `allow_synthetic_fallback: false` in `config/config.yaml`, delete `data/raw/`, and re-run `python run.py evaluate`. Then update the figures in Section XVI of the report. The framework records this distinction on the dataset node precisely so it cannot be missed.

10. Glossary of every term used

Append-only

A store that permits insertion but not modification or deletion. Here it is enforced by database triggers, so the guarantee holds against any program, not just well-behaved ones. The reason is evidential: a record that can be quietly edited proves nothing.

Artifact

Any named, versioned thing the lifecycle produces — a dataset, a feature set, a model, a deployment, a drift report. In this system every artifact is a node in the graph.

Average precision (PR-AUC)

The area under the precision–recall curve. It is the primary metric here because the positive class is only 3.39% of the data, and PR-AUC — unlike accuracy or ROC-AUC — does not flatter a model that mostly predicts "no failure".

Class imbalance

When one outcome is far rarer than the other. Here failures are 3.39% of rows. Imbalance makes accuracy useless: a model predicting "never fails" scores 96.6%.

Checksum / digest

A short fingerprint computed from the bytes of a file or the values of a table, using SHA-256. If one value changes, the digest changes completely. This is how the system knows two datasets are or are not the same.

Code revision

The exact state of the source code when a run executed — a git commit hash, or a hash of the source tree if git is not in use.

Concept drift / data drift

When the data arriving in production stops resembling the data the model was trained on. Here it is measured with PSI. Drift is why a model that worked last year may be wrong today.

Content addressing

Identifying something by a hash of its content rather than by a name or a counter. Two artifacts with the same content get the same identifier automatically; two with different content cannot collide. This is what makes the reproducibility claim checkable.

Data lineage

The record of where data came from and what was done to it. Often used interchangeably with provenance; lineage tends to emphasise the data path, provenance the full causal history.

DataOps

Applying engineering discipline — version control, testing, automation, monitoring — to data pipelines. The data-side counterpart of MLOps.

Deployment

A model that has been promoted into service. Recorded here as its own artifact carrying the gate it passed and the metric that justified promotion.

Digital thread

The central concept of the project. A continuous, authoritative record linking every representation of a product across its whole lifecycle, so any representation can be traced to any other.

Digital twin

A simulation model of a physical thing. Frequently confused with the digital thread. The twin is a model of the thing; the thread is the record of how the thing came to be.

DVC (Data Version Control)

A tool that content-addresses datasets and models so they can be versioned alongside code in git. This project records DVC-style digests itself and can sit alongside DVC without depending on it.

Edge

A typed link between two artifacts in the graph, such as DERIVED_FROM. Every edge in this system points backwards in time.

Environment digest

A fingerprint of the interpreter and library versions a run executed in. Needed because the same code and data can produce different results under different library versions.

F1 score

The harmonic mean of precision and recall, giving a single number that punishes a model that is good at one and bad at the other.

Feature

An input column the model actually learns from. Here, three are engineered from the documented physics of machine failure rather than taken raw.

Feature engineering

Constructing model inputs from raw measurements. The temperature difference, mechanical power and the wear–torque product are all engineered features here.

Feature store

A system that manages feature definitions and keeps training and serving computation consistent. This project stores feature definitions in the ledger, achieving the consistency property without a separate store.

Holdout split

Reserving part of the data, untouched during training, to test the model on. Here 20% is held out, stratified so the rare failure class appears in both halves.

Idempotent

An operation that can be repeated without changing the outcome. Writing the same edge twice here returns the existing edge rather than creating a duplicate.

Impact analysis

The forward query: given an artifact, what else was affected by it? The query you need when a defect is found in source data and you must establish the blast radius.

Ledger

The name given here to the append-only graph store. "Ledger" rather than "database" because the defining property is that entries are added and never altered.

Logistic regression

A simple linear classifier. Used here as the weaker of two candidates — it achieves high recall by flagging far too much, with precision of only 0.094.

MLflow

A widely used experiment tracking platform. This project records MLflow run identifiers as plain attributes, so the two interoperate without a dependency.

MLOps

The practice of operating machine learning systems in production: versioning, deployment, monitoring and retraining.

Model registry

A catalogue of trained models and their promotion status. Here the `deployment` node plays this role inside the graph itself.

Node

An artifact in the graph, carrying a type, a name, a version, a content digest and arbitrary attributes.

Orchestration

Scheduling pipeline stages, resolving their dependencies and retrying failures. Prefect and Airflow are the common tools. Deliberately kept separate from lineage here, so the record does not belong to the scheduler.

Parquet

A compressed columnar file format, used for intermediate datasets because it preserves types and reads faster than CSV.

Precision

Of the cases the model flagged, the proportion that really were failures. Low precision means false alarms.

Promotion gate

A minimum score a candidate must reach before it may be deployed. Recorded with the deployment, so the reason for promotion is part of the record.

Provenance

The full causal history of an artifact: what it was derived from, by what process, under what conditions.

PROV (W3C)

A published standard for describing provenance. Its key contribution here is the convention that relations point backwards in time, which this project adopts wholesale.

PSI (Population Stability Index)

A statistic measuring how far a distribution has moved from a reference. Conventionally, below 0.10 is stable, 0.10–0.25 warrants attention, and above 0.25 is a material shift. The threshold here is 0.20.

Random forest

An ensemble of decision trees voting together. The stronger of the two candidates here, and the one promoted.

Recall

Of the failures that actually occurred, the proportion the model caught. Low recall means missed failures.

Reproducibility

The ability to obtain the same result again from the same inputs. In this project it is measured rather than assumed, by comparing content-addressed artifact identities across five independent runs.

SHA-256

The cryptographic hash function used for all digests. Chosen because an accidental collision is effectively impossible, so equal digests mean equal content.

SQLite

A database that lives in a single file with no server. Chosen so the entire lineage record is one portable, archivable artifact.

Target leakage

When a feature secretly contains the answer. Here the five per-mechanism failure flags are components of the target; leaving them in would produce a near-perfect model that has learned nothing. They are dropped, and the fact that they were dropped is recorded.

Traceability

The property that any artifact can be followed to its origins and its consequences. The central claim this project measures.

Traversal (upstream / downstream)

Walking the graph. Upstream follows edges outwards to recover provenance; downstream follows them inwards to recover impact.

Trigger (database)

A rule the database itself enforces when a row is changed. Used here to refuse updates and deletes, making the append-only guarantee structural rather than conventional.

Validation / expectation suite

A set of rules the data must satisfy — required columns, row count, null and duplicate limits, value ranges. The verdict is stored as its own artifact, so "was this data checked?" is answerable later.

Wall clock

Elapsed real time, as opposed to processor time. All timings reported here are wall clock.

11. Questions you should be ready for

"Isn't this just MLflow and DVC glued together?"

No, and the comparison figure in the report makes the distinction precisely. MLflow records experiments; DVC versions files. Neither records data validation verdicts, feature definitions, serving events or drift observations as part of the same graph, and neither can be traversed in both directions with one query. The contribution is the joining abstraction, not the tools.

"Can it tell you which rows caused a prediction?"

No. Lineage is recorded at artifact granularity — this dataset version, this feature set, this model. Row-level provenance is substantially harder and is explicitly out of scope. This is the first limitation stated in the report, and stating it first is deliberate.

"Will it scale?"

Unknown, honestly. It was evaluated on a 32-node graph. SQLite and in-memory traversal are comfortable there and will be for a long time, but the crossover point where a dedicated graph store earns its place depends on query shape, not just node count, so it has to be measured rather than extrapolated. That benchmark is the first item on the near-term roadmap.

"What is the 100% reproducibility actually claiming?"

That with inputs, code and seed held constant, the content-addressed identities of every dataset, feature set and model match across five independent runs. It is not a claim that machine learning is deterministic. Its value is that any future divergence becomes visible instead of silent.

"Why is the overhead 40% when you say the ledger costs 1.9%?"

Because they measure different things, and both are reported for that reason. 1.9% is time spent writing lineage. The remaining difference is persisting intermediate artifacts and profiles that the baseline computes in memory and discards — a precondition of traceability, but not a cost of the ledger. Also note the pipeline is dominated by a 3.06-second model fit, so the ratio is a property of a deliberately small case study.

"Were these results produced on the real dataset?"

Not in the run reported. The execution environment could not reach the UCI repository, so a replica was generated from the dataset's published description, matching its documented distributions and its 3.39% failure rate. The ledger records this as an `origin` attribute on the dataset node and the dashboard shows it in a warning colour. Regenerate against the real file before citing the modelling results.